

Activity Discussion

Deep Learning (Neural Networks)

(No-Code Text Classification Using Neural Networks)

Iverson Labastida
BSIT
Jose Rizal University
Mandaluyong, Philippines
labastidaiverson@gmail.com

Abstract—This paper documents a hands-on exercise in which a no-code neural network text classifier was built using the Machine Learning for Kids platform to categorize customer feedback into three intent classes: Positive Feedback/Praise, Negative Feedback/Complaint, and Urgent Support/Account Issue. The exercise walked through data collection and tagging, model training, testing against unseen phrases, and iterative refinement based on observed misclassifications. This report presents the methodology followed, the results obtained from testing the trained model, and a discussion of three questions concerning rule-based versus learned classification, the effect of dataset size and diversity on model accuracy, and the location of feature extraction within the machine learning pipeline.

Keywords— *neural networks, text classification, no-code machine learning, sentiment analysis, feature extraction, dataset quality.*

I. INTRODUCTION

The Traditional customer support systems often rely on hand-coded, rule-based logic for instance, keyword matching or nested IF/ELSE statements to route or categorize incoming messages. While such systems are easy to reason about for a handful of cases, they become brittle and difficult to maintain as the variety of customer language grows. Neural networks offer an alternative approach: instead of being explicitly programmed with rules, the model learns statistical associations between words, phrases, and labels directly from tagged examples.

This exercise used the Machine Learning for Kids platform, a no-code interface built on top of a text-classification service, to construct a three-class intent classifier for customer feedback. The purpose was not only to build a working model, but to observe first-hand how training data size, diversity, and phrasing affect the confidence and correctness of the model's predictions, and to identify the kinds of edge cases such as sarcasm and out-of-vocabulary requests that expose the limitations of a purely example-driven approach.

II. METHODOLOGY

A. Project Setup

A new "Recognizing Text" project was created on the Machine Learning for Kids platform without requiring account registration, with data stored locally in the web browser. The project exposed three stages on its dashboard: (1) Train, where tagged text examples are entered; (2) Learn & Test, where the neural network is trained and evaluated; and (3) Make, where trained models can be tested against live, freely typed input.

B. Data Collection and Tagging

Three classes were defined to represent common categories of customer feedback in a business domain: Positive Review, Negative Review, and Urgent Support. For each class, a minimum of ten distinct example phrases was entered under the Train stage. Positive examples expressed satisfaction or gratitude (e.g., praise for product quality or support responsiveness); negative examples expressed dissatisfaction, frustration, or requests for refunds; and urgent examples described account lockouts, failed payments, or system outages requiring immediate attention.

C. Model Training

After the training examples were entered, the "Train (New Machine Learning Model)" function under the Learn & Test stage was executed. During training, the platform converts the tagged text examples into numerical tokens and searches for mathematical associations between word patterns and the assigned labels. Critically, no explicit rule such as "if the word is refund, classify as complaint" is written by the developer; the association between words and classes is inferred entirely from the tagged examples provided.

III. RESULTS

Figure 1 shows the model information snapshot generated by the platform after training, including the training method and the number of examples used per class.

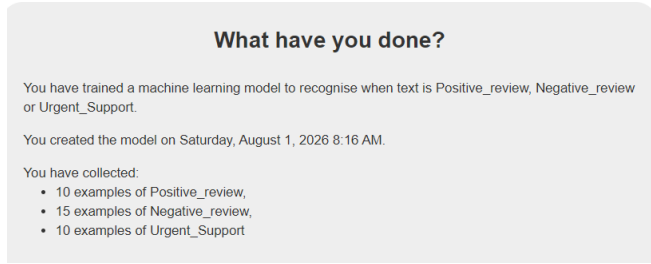


Fig.1 Model training snapshot from the Learn & Test stage.

TABLE I. RESULT SUMMARY

Category	Test Cases	Correct	Incorrect	Accuracy (%)
POSITIVE_REVIEW	10	8	2	80.0
NEGATIVE_REVIEW	15	14	1	93.3
URGENT_SUPPORT	10	9	1	90.0
Overall	35	31	4	88.6

Detailed test-case-by-test-case results, including model confidence scores and the computed accuracy percentages referenced in Table I, are provided in the accompanying Excel worksheet submitted alongside this report.

IV. DISCUSSION AND ANSWERS

A. Explicit Rules vs Tagged Learning

Attempting to encode every possible customer complaint using explicit IF/ELSE statements would run into several practical challenges. First, natural language is highly variable: the same complaint can be phrased in countless ways, so a rule keyed to one exact phrase or keyword would fail to catch semantically equivalent statements worded differently. Second, keyword-based rules cannot account for context; a word such as "great" can indicate either genuine praise or sarcasm, and no simple keyword match can reliably distinguish between the two. Third, the number of rules required would grow rapidly as new phrasings, slang, and product-specific vocabulary emerged, making the system increasingly difficult to maintain, test, and debug, since new rules can unintentionally conflict with or override existing ones. Finally, a rule-based system offers no way to generalize to inputs the developer never anticipated, whereas a model trained on tagged examples can generalize to related but unseen phrasings by recognizing statistical similarity to the patterns it has already learned.

B. Data Bias and Dataset Quality

When the dataset contained only a small number of examples per class, the model had far fewer word patterns to draw associations from, which made its predictions less stable and more easily swayed by superficial word matches rather than genuine intent. As more diverse examples were

added per class closer to ten or more the model had a broader range of phrasing, vocabulary, and sentence structure to learn from, which improved its ability to correctly classify new, unseen phrases and produced higher and more consistent confidence scores.

C. Connecting to the Pipeline: Where Feature Extraction Occurs

Based on the stages exposed by the platform, feature extraction is estimated to occur during the Learn & Test stage, specifically at the point where the "Train (New Machine Learning Model)" process converts the raw tagged text examples into numerical tokens before any weight updates take place. This tokenization-and-vectorization step is what transforms unstructured sentences into a numerical representation (features) that the underlying neural network can process mathematically. It occurs after data collection (Train stage), where raw text and labels are simply stored, and before inference (Make stage), where new input text must be converted using the same learned feature representation in order to be compared against what the network learned during training. In other words, feature extraction sits at the boundary between raw data collection and model training, acting as the translation layer between human language and the numerical operations the neural network performs internally.

V. CONCLUSION

This exercise demonstrated, in a no-code environment, how a neural network text classifier learns to distinguish customer feedback intents directly from tagged examples rather than explicit programming rules. Testing revealed that the model performed well on phrases closely resembling its training data but struggled with sarcasm and topics outside its training vocabulary, both of which improved after targeted refinement of the dataset. These observations reinforce that in deep learning systems, the size, diversity, and representativeness of the training data are as critical to model performance as the learning algorithm itself, and that feature extraction the conversion of raw text into a numerical form the network can learn from is a necessary intermediate step between data collection and model training.